

METADATA

Field	Value
Incident ID	QSM-2026-SQLI-001
Report author	Anna-Kayli Bradley
Role	SOC analyst, ITS25
Operators (fictional)	QuickSilver Materials
Sector under the Cybersecurity Act	Chemicals and Production Distribution
Date of the report	2026-05-31
The report version	1.0
Document status	Final report

A1. SUMMARY

Between 11–13 May a suspicious activity was identified on Wazuh against the QuickSilver Materials web services. The activity originated from internal Docker source IP 172.18.0.1 and targeted the main app hosted behind the nginx-proxy container on 172.18.0.7.

Initial indicators displayed repeated abnormal HTTP 400/500 responses against potentially sensitive APIs like /rest/user/login, /rest/admin/application-configuration, /rest/user/whoami, multiple login failures, probing of admin-related APIs and multiple- user and financial-related information. Over the following days the activity became more targeted and progressed from reconnaissance-style action to eventually successful access to admin account and retrieval of customer information.

The investigation uncovered repetitive interaction with functions related to login, user identity, addresses, payment card details, delivery information, basket contents and admin configuration functionality. The sequence of events present a strong case against possible SQL injection and abuse of broken access control as a result of privileged access.

Analysis was conducted primarily through Wazuh-generated alerts, nginx logs, Docker container inspection and real-time monitoring through interactive shell to prove/disprove the SQL hypothesis. Although the logs provided clear visibility into request flow, specific API targeting and attacker's behaviour, the logs didn't capture SQL payload or backend database queries. As a result, the investigation relied on behavioural analysis, HTTP patterns and chronology reconstruction rather than direct payload presence.

The incident presents no other evidence of compromise beyond the above mentioned attack. It is reasonable to expect however that the accessed customers data may be vulnerable to further exploitation beyond the scope of this investigation.

A2. PREPARATION

2.1 Organisation description

QuickSilver Materials is a fictional company operating within the Chemicals and Production Distribution sector under the Cybersecurity Act (NIS2).

The company develops high-performance engineered material solutions optimized for precision manufacturing, industrial efficiency, and global supply integration.

Due to its role in industrial production and supply-chain operations, the organization is considered part of a critical sector requiring cybersecurity monitoring and incident reporting.

2.2 Security Monitoring Environment

The incident investigation was conducted in a Wazuh-based local lab environment running through Docker containers. The environment consisted of the following containers (only relevant to this report):

- Wazuh Manager → collects logs from agents, applies rules and generates alerts
- Wazuh Dashboard → web interface visualising all the Manager accumulates

- Wazuh Indexer → allows for keeping and searching for events in the past
- nginx reverse proxy agent → forwards traffic from Juice Shop app to the nginx container and generates logs on behalf of Juice Shop
- OWASP juice shop agent → provides the “attack” surface, contains the app only. Further referred to as “web services” in terms of company’s assets.

2.3 Log sources

For the purpose of better understanding of the mechanics of this exercise multiple sources of logs and background information were accessed for monitoring and analysis. The primary sources include:

- Wazuh’s own generated events, accessed through **localhost**
- Docker container logs → accessed through: **docker ps & docker logs <container number>**
- nginx logs (**access.log** and **error.log**) through **root@nginx-proxy**
- Linux interactive container shell to monitor events in real time → accessed through: **docker exec -it logghantering-docker-nginx-proxy-1 bash** (within the “logghantering” dir) and **tail -f /var/log/nginx/access.log**
- Docker core data at **/var/lib/docker** (for troubleshooting and deeper understanding only)

Wazuh agents forwarded log events to the Wazuh manager for central analysis and alert generation as the attacks occurred in real time.

(Table 1 shows hashed main log sources with system timestamps. The timestamps were collected using cmd **stat <file>**)

File name	Location	256sha #	Source	Timestamp
access.log	/var/log/nginx/ access.log	11fb4465f4fd4daaa776b3fc36f53450f 95e067ba492187e3f28162d008a3311	nginx-proxy	Access: 2026-05-24 20:37:53.954916930 +0000 Modify: 2026-05-24 20:37:53.867915184 +0000 Change: 2026-05-24 20:37:53.867915184 +0000 Birth: 2026-05-09 19:08:09.831675541 +0000
error.log	/var/log/nginx/ error.log	e3b0c44298fc1c149afb4c8996fb9242 7ae41e4649b934ca495991b7852b855	nginx-proxy	Access: 2026-05-24 15:42:07.500017020 +0000 Modify: 2026-05-21 17:27:27.503000940 +0000 Change: 2026-05-21 17:27:27.503000940 +0000 Birth: 2026-05-09 19:08:09.832913642 +0000
wazuh-alerts.csv	/log-assignment/ logs/wazuh-alerts.csv	3555fdf287f3c167de516beb7dc6424fd 36a328c95bf14bb88053e58d761be8b	Wazuh	Access: 2026-05-25 15:30:42.405354801 +0200 Modify: 2026-05-25 15:25:30.015590952 +0200 Change: 2026-05-25 15:28:20.821611562 +0200 Birth: 2026-05-25 15:25:30.012590918 +0200

2.4 Wazuh’s architecture

The Wazuh architecture consists of three major Wazuh containers:

1. Wazuh Manager that aggregates logs, integrates them into a coherent representation of the events, applies rules to these events and generates alerts as interprets them.
2. Wazuh Indexer that holds and indexes all passed events for future view and analysis.
3. Wazuh Dashboard that displays all the logs and data in a visual way.

The agents are another set of containers that are active in the system and act as log sources and activity platforms forwarding telemetry and event logs to the manager.. The two containers that are relevant to the task are:

4. The Juice Shop container serves as an attack surface where the SQL injection, broken access credential and data exfiltration attacks take place
5. The nginx proxy container forwards traffic from the original Juice Shop container that acts as an attack surface, generates web access and forwards respective logs to Wazuh.

Main log data was collected from container log outputs in Wazuh, with auth.log being the main source of intel. Auth.log entries from nginx native directory were forwarded to the Wazuh Manager where they were parsed and mapped to pinpoint rules for security monitoring and alert generation. Other internal log files in the nginx system dir such as syslog, user.log, error.log were accessed through the terminal on the host machine and served as a supplemental data to observe how the events develop in real time.

During the investigation two IP sources will become of primary importance:

172.18.0.7- Docker container's IP adapted by nginx-proxy agent that serves as an attack "victim"

172.18.0.1- source IP that sends the HTTP requests to nginx. "The attacker" or Docker host gateway, which is essentially the host machine communicating with the containers.

2.5 Objective of the exercise

The objective of the exercise was to simulate a realistic SQL injection, broken access control attack and data exfiltration through OWASP Juice Shop, investigate the incident from an analyst perspective, and document the incident response process according to the SANS outlay.

2.6 Rule sets

Access point: **docker exec -it fe074c55b2ea bash** (Wazuh Manager container) → **cd /var/ossec/ruleset/rules**

Wazuh Manager was configured to load the default Wazuh ruleset from **/var/ossec/ruleset/rules**, including rule files such as **0245-web_rules.xml**, **0260-nginx_rules.xml**, and **0280-attack_rules.xml**. During the investigation, Wazuh mainly created alerts from nginx logs. It used built-in rules that treat HTTP 400 and 500 responses as possible attack signals. These were mainly matched by rule 31101 (400 errors) and rule 31122 (500 errors).

(Screenshot 1 shows predefined rule definitions used to detect and classify events from incoming logs)

```
bash-5.2# cd /var/ossec/ruleset/rules
bash-5.2# ls
0010-rules_config.xml      0175-proftpd_rules.xml      0345-netscaler_rules.xml    0515-exim_rules.xml         0705-sophos_fw_rules.xml
0015-ossec_rules.xml       0180-pure-ftpd_rules.xml     0350-amazon_rules.xml       0520-vulnerability-detector_rules.xml  0715-freebx_rules.xml
0016-wazuh_rules.xml       0185-vsftpd_rules.xml       0360-serv-u_rules.xml       0525-openvas_rules.xml      0750-github_rules.xml
0017-wazuh-api_rules.xml   0190-ms_ftpd_rules.xml      0365-auditd_rules.xml       0530-mysqld_audit_rules.xml  0755-office365_rules.xml
0020-syslog_rules.xml      0195-named_rules.xml        0375-usb_rules.xml          0535-mariadb_rules.xml      0770-gitlab_rules.xml
0025-sendmail_rules.xml    0200-smbd_rules.xml         0380-redis_rules.xml        0540-pfsense_rules.xml      0775-arbor_rules.xml
0030-postfix_rules.xml     0205-racoon_rules.xml       0385-oscaped_rules.xml      0545-osquery_rules.xml      0780-fireeye_rules.xml
0035-spamd_rules.xml       0210-vpn_concentrator_rules.xml  0390-fortiddos_rules.xml    0550-kaspersky_rules.xml    0785-huawei-usg_rules.xml
0040-inapd_rules.xml       0215-policy_rules.xml       0391-fortigate_rules.xml    0555-azure_rules.xml        0800-sysmon_id_1.xml
0045-mailscanner_rules.xml 0220-msauth_rules.xml       0392-fortimail_rules.xml    0560-docker_integration_rules.xml      0810-sysmon_id_3.xml
0050-ms_exchange_rules.xml 0225-mcafee_av_rules.xml     0393-fortiauth_rules.xml    0565-ms_ipsec_rules.xml     0820-sysmon_id_7.xml
0055-courier_rules.xml     0230-ms-se_rules.xml        0395-hp_rules.xml           0570-sca_rules.xml          0830-sysmon_id_11.xml
0065-pix_rules.xml         0235-vmware_rules.xml       0400-openvpn_rules.xml      0575-win-base_rules.xml     0840-win_event_channel.xml
0070-netscreenfw_rules.xml 0240-ids_rules.xml           0405-rsa-auth-manager_rules.xml 0580-win-security_rules.xml  0850-audit_rules.xml
0075-cisco-tos_rules.xml   0245-web_rules.xml           0410-imperva_rules.xml      0585-win-application_rules.xml 0860-sysmon_id_13.xml
0080-sonicwall_rules.xml   0250-apache_rules.xml        0415-sophos_rules.xml       0590-win-system_rules.xml    0870-sysmon_id_8.xml
0085-pan_rules.xml         0255-zeus_rules.xml          0420-freeipa_rules.xml      0595-win-sysmon_rules.xml    0900-firewall_rules.xml
0090-telnetd_rules.xml     0260-nginx_rules.xml         0425-cisco-estreamer_rules.xml 0600-win-wdefender_rules.xml 0905-cisco-ftd_rules.xml
0095-sshd_rules.xml        0265-php_rules.xml           0430-ms_wdefender_rules.xml 0601-win-vipre_rules.xml     0910-ms-exchange-proxylogon_rules.xml
0100-solaris_bsn_rules.xml 0270-web_appsec_rules.xml    0435-ms_logs_rules.xml      0602-win-wfirewall_rules.xml 0915-win-powershell_rules.xml
0105-asterisk_rules.xml    0275-squid_rules.xml         0440-ms_sqlserver_rules.xml 0605-win-mcafee_rules.xml    0920-oracledb_rules.xml
0110-ms_dhcp_rules.xml     0280-attack_rules.xml        0445-identity_guard_rules.xml 0610-win-ms_logs_rules.xml   0925-eset-remote_rules.xml
0115-arpwatch_rules.xml    0285-systemd_rules.xml       0450-docker_rules.xml       0615-win-ms-se_rules.xml     0935-cloudflare-waf_rules.xml
0120-symantec-av_rules.xml 0290-firewall_rules.xml      0455-mongodb_rules.xml      0620-win-generic_rules.xml    0945-sysmon_id_10.xml
0125-symantec-ws_rules.xml 0295-mysql_rules.xml         0460-jenkins_rules.xml      0625-cisco-asa_rules.xml     0950-sysmon_id_20.xml
0130-trend-osce_rules.xml  0300-postgresql_rules.xml    0470-vshell_rules.xml       0625-mcafee_epo_rules.xml    0955-WEF-baseline_rules.xml
0135-hordeimp_rules.xml    0305-dropbear_rules.xml      0475-suricata_rules.xml     0630-nextcloud_rules.xml     0960-macos_rules.xml
0140-roundcube_rules.xml   0310-openbsd_rules.xml       0480-qualysguard_rules.xml   0635-owlh-zeek_rules.xml     0990-amazon-security-lake_rules.xml
0145-wordpress_rules.xml   0315-apparmor_rules.xml      0485-cylance_rules.xml       0640-junos_rules.xml         0995-microsoft-graph_rules.xml
0150-cinserver_rules.xml   0320-clam_av_rules.xml       0490-virustotal_rules.xml    0675-panda-paps_rules.xml    0997-maltiverse_rules.xml
0155-dovecot_rules.xml     0325-opensmtpd_rules.xml     0495-proxmox-ve_rules.xml    0680-checkpoint-smarti_rules.xml 0998-aws-security-hub_rules.xml
0160-vmpop3d_rules.xml     0330-sysmon_rules.xml        0500-owncloud_rules.xml     0690-gcp_rules.xml           0999-malicious-ioc-rules.xml
0165-vpopmail_rules.xml    0335-unbound_rules.xml       0505-vuls_rules.xml         0695-f5-bigip_rules.xml
0170-ftp_rules.xml         0340-puppet_rules.xml        0510-ciscat_rules.xml       0700-paloalto_rules.xml
```

A3. IDENTIFICATION

3.1 First indication

The first indication of suspicious activity was marked on the 11th of May through Wazuh-generated alerts on the nginx-proxy agent. The alerts were detected in /var/log/nginx/access.log and appeared as abnormal requests targeting privileged API endpoints.

(Screenshot 2 shows alerts and relevant rules generated by Wazuh upon conducting attacks on Juice Shop)

	timestamp	agent.name	rule.description	rule.level	rule.id
	May 13, 2026 @ 17:23:13.1...	nginx-proxy	Web server 400 error code.	5	31101
	May 13, 2026 @ 17:23:13.1...	nginx-proxy	Web server 400 error code.	5	31101
	May 13, 2026 @ 17:23:13.1...	nginx-proxy	Web server 400 error code.	5	31101
	May 13, 2026 @ 17:16:08.9...	nginx-proxy	Wazuh agent started.	3	503
	May 12, 2026 @ 21:15:31.7...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 21:14:43.7...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 21:12:05.6...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 21:09:41.7...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 21:02:53.6...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 21:01:59.6...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 20:49:59.5...	nginx-proxy	Web server 400 error code.	5	31101
	May 12, 2026 @ 20:49:55.5...	nginx-proxy	Web server 400 error code.	5	31101
	May 12, 2026 @ 20:48:47.5...	nginx-proxy	Web server 400 error code.	5	31101
	May 12, 2026 @ 20:29:55.4...	nginx-proxy	Web server 400 error code.	5	31101
	May 12, 2026 @ 20:28:37.4...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122

Rows per page: 15

Initial investigation through Wazuh log assessed that the first alert appeared at 18.37 and last one at 22.02. Together 40 alerts were generated during the interval of 3 hours and 25 minutes with an average rate of 11.7 alerts per hour/ 1 alert per 5.1 minute. The alerts collectively consisted of 18 400-HTTP errors (client-side) and 22 500-HTTP errors (server-side).

(Screenshot 4 illustrates the entire chain of events from Day 1, the 11th of May)

	May 11, 2026 @ 22:02:40.2...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 22:02:32.2...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 22:01:38.2...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 22:00:38.2...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 22:00:18.2...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 21:59:58.2...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:59:42.2...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:59:22.2...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:55:04.2...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 21:47:36.1...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:45:42.1...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:45:34.1...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:44:50.1...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 21:44:42.1...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 21:44:32.1...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 21:44:24.1...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 21:44:06.1...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 21:41:46.1...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:40:36.1...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 21:40:28.1...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:39:04.1...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:39:00.1...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:38:46.1...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:37:28.1...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 21:36:36.1...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:36:32.1...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:33:38.0...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:33:32.0...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:33:24.0...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:33:14.0...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 21:33:04.0...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122	-
	May 11, 2026 @ 20:08:19.4...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 19:40:33.2...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 19:40:27.2...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 18:54:34.9...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 18:52:18.9...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 18:50:50.9...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 18:47:32.9...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 18:47:12.9...	nginx-proxy	Web server 400 error code.	5	31101	-
	May 11, 2026 @ 18:37:30.9...	nginx-proxy	Web server 400 error code.	5	31101	-

Table	JSON
† _index	wazuh-alerts-4.x-2026.05.11
† agent.id	002
† agent.ip	172.18.0.7
† agent.name	nginx-proxy
† data.id	401
† data.protocol	POST
† data.srcip	172.18.0.1
† data.url	/rest/user/login
† decoder.name	web-accesslog
† full_log	172.18.0.1 - - [11/May/2026:18:37:29 +0000] "POST /rest/user/login HTTP/1.1" 401 26 "http://172.0.0.1:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0"
† id	1778524650.0
† input.type	log
† location	/var/log/nginx/access.log
† manager.name	wazuh.manager
† rule.description	Web server 400 error code.
# rule.firedtimes	1
† rule.gdpr	IV_35.7.d
† rule.groups	web, accesslog, attack
† rule.id	31101
# rule.level	5
⊗ rule.mail	false
† rule.nist_800_53	SA.11, SI.4
† rule.pci_dss	6.5, 11.4
† rule.tsc	CC6.6, CC7.1, CC8.1, CC6.1, CC6.8, CC7.2, CC7.3
⊞ timestamp	May 11, 2026 @ 18:37:30.912

The evidence to the left (screenshot 5) shows the first alert generated by nginx-proxy access logs and it displays a failed POST request against /rest/user/login API. The request returned 401 Unauthorised.

The request originated from the internal Docker network source 172.18.0.1 and was classified by Wazuh rule 31101 as abnormal web activity. At this stage, the event alone didn't classify as malicious.

Further investigation reveals however that on the day of 11th May IP 172.18.0.7 received numerous GET and POST request from the same IP exploring access attempts to various API endpoints such as rest/user/login, rest/user/admin, rest/admin/application-configuration/author, rest/savedLoginIp etc,

Although the default Wazuh rules didn't explicitly define the activity as a specific attack, the timing and sensitivity of the targeted API endpoints suggested suspicious interaction with the app. The alerts generating 400 errors can reasonably indicate access

probing and recon activity and 500 are related to backend and API exploitation attempts.

On the day of 12th May 11 alerts have been generated on nginx-proxy agent. 4 of them were 400 errors, the rest were 500 errors. Closer inspection revealed much more precise API targeting, especially multiple POST requests at /rest/user/login and /profile APIs. All request came from the same IP as the day before. Targeted interaction with repeated login objects changed previous hypothesis to authentication probing.

On the day of 13th May Wazuh generated only 3 400 alerts showing the same IP targeting very specific APIs related to users personal data such as api/Address/null, api/Cards/null, api/Deliveries/null (further screenshot in the Chronology section)

(Screenshot 6 illustrates the entire chain of events from the final day, the 13th of May)

4 hits					
May 13, 2026 @ 00:00:00.000 - May 13, 2026 @ 23:30:00.000					
⏏ Export Formatted	⏏ Reset view	⚙ 638 available fields	⌵ Columns	⌵ Density	⚙ 1 fields sorted
⏏ Full screen					
timestamp	agent.name	rule.description	rule.level	rule.id	data.ms-graph.deviceName
May 13, 2026 @ 17:23:13.1...	nginx-proxy	Web server 400 error code.	5	31101	-
May 13, 2026 @ 17:23:13.1...	nginx-proxy	Web server 400 error code.	5	31101	-
May 13, 2026 @ 17:23:13.1...	nginx-proxy	Web server 400 error code.	5	31101	-
May 13, 2026 @ 17:16:08.9...	nginx-proxy	Wazuh agent started.	3	503	-

A3.2 Chronology

The activity observed in Wazuh logs spans three distinct phases between 11–13 May. All events originate from the same internal Docker source IP 172.18.0.1 targeting the nginx-proxy agent and the web services APIs hosted on 172.18.0.7.

On 11th May the first wave of activity consisted mainly of forced-entry attempts and initial API probing. These requests produced a mixture of HTTP 400 and 500 errors indicating multiple failed login attempts. Over a period of around 3 hours, repeated requests were observed against multiple sensitive objects like user, admin and login-related APIs. This day specifically reflects what looks like initial reconnaissance and access testing attempts. Deeper dive into access.log on the nginx-proxy agent reveals example of suspicious behaviour through the following chain of events:

(Table 2 shows the chronology of events from the 11th of May: first Wazuh-generated alert and then matching event from the nginx access.log)

Timestamp	Source	Event	Interpretation
11-05 21:40	Wazuh-generated alert	Wazuh sends HTTP 500	requests were interacting with the backend, potentially triggering errors during probing or exploitation attempts
11-05 21:40	nginx access.log (Wazuh matching event)	request to /rest/admin/image returned HTTP 500	someone's interacting with an admin-governed API, suggesting abnormal input or possible misuse of privileged information
11-05 21:40	Wazuh generated alert	Wazuh sends 400 error code	malformed, unexpected, or suspicious client-side interaction with the web service
11-05 21:40	nginx access.log (Wazuh matching event)	request to /rest/saveLoginIp returned HTTP 401	illegitimate attempt to access functionality linked to account control
11-05 21:40	Wazuh generated-alert	Wazuh sends 500 alert	possible probing or exploitation attempts
11-05 21:41	nginx access.log (Wazuh matching event)	request to admin-related object returned HTTP 500	possible backend manipulation or inaccurate request affecting server behaviour
11-02 21:42	nginx access.log (no matching event passed in Wazuh)	successful (200) request sent to rest/admin/application-version.	session was able to successfully interact with an admin-related API and retrieve internal information normally not intended for ordinary users

(Screenshot 7 shows raw log adequately representing the chronology described in Table 2)

```

172.18.0.1 - - [11/May/2026:21:40:27 +0000] "GET /rest/admin/image HTTP/1.1" 500 757 "-" Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
172.18.0.1 - - [11/May/2026:21:40:29 +0000] "GET /favicon.ico HTTP/1.1" 200 9346 "http://127.0.0.1:3000/rest/admin/application-configuration" Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
172.18.0.1 - - [11/May/2026:21:40:36 +0000] "GET /rest/saveLoginIp HTTP/1.1" 401 12 "-" Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
172.18.0.1 - - [11/May/2026:21:40:57 +0000] "GET /assets/t18n/en.json HTTP/1.1" 200 11967 "http://127.0.0.1:3000/rest/admin/application-configuration" Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
172.18.0.1 - - [11/May/2026:21:40:57 +0000] "GET /favicon.ico HTTP/1.1" 200 9346 "http://127.0.0.1:3000/assets/t18n/en.json" Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
172.18.0.1 - - [11/May/2026:21:41:45 +0000] "GET /rest/admin/application-version HTTP/1.1" 200 20 "-" Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
172.18.0.1 - - [11/May/2026:21:41:48 +0000] "GET /favicon.ico HTTP/1.1" 200 9346 "http://127.0.0.1:3000/rest/admin/application-configuration" Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
172.18.0.1 - - [11/May/2026:21:42:35 +0000] "GET /rest/admin/application-version HTTP/1.1" 200 20 "-" Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
172.18.0.1 - - [11/May/2026:21:42:35 +0000] "GET /favicon.ico HTTP/1.1" 200 9346 "http://127.0.0.1:3000/rest/admin/application-version" Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0

```

What's interesting to see is that Wazuh doesn't generate HTTP 200 alerts upon successful login, which causes a significant setback in the interpretation of events if one relies on Wazuh alerting indiscriminately. At this point it's reasonable to expect that the system is under recon and very possibly under indirect attack.

On 12 May, activity resumed with a lower volume but much more precise targeting of login and user objects. Requests again focused on /rest/user/login and /rest/user/whoami, with alternating successful (200) and failed (401/500) responses. The pattern suggests iterative testing of credentials or session validation behaviour rather than random probing. The activity became more focused compared to the first day, with repeated interaction on a smaller set of endpoints. The screenshot (8) shows a failed login attempt that triggered a server error (POST /rest/user/login → 500), followed immediately by a successful login (200) and stable accessed session confirmed by repeated /rest/user/whoami responses. The raw log from between the first (20:28) and last (21:15) Wazuh generated alert consists of nearly 200 entries hence only a small example of 6 entries is exemplified in this report within the alert window to illustrate progression in the attack mindset. The sequence from given example suggests forced-access attempts around the login function, after which the session resolves successfully.

(Table 3 shows the chronology of day 2)

Timestamp	Source	Event	Interpretation
12-05 21:09	Wazuh	Wazuh generates HTTP 500	indicative of probing (as per consecutive actions soon after)
12-05 21:09	nginx access.log	POST /rest/user/login returned HTTP 500	login attempt triggered an internal server error, suggesting incorrect input or backend meltdown during access
12-05 21:09	nginx access.log	GET /rest/user/whoami requests returned HTTP 200	user identity checks were done before and after the failed login
12-05 21:10	nginx access.log	POST /rest/user/login returned HTTP 200	account login successful after the series of earlier failed login attempts
12-05 21:10	nginx access.log	GET /rest/user/whoami requests returned HTTP 200	session validation continued after successful login, confirming active state

(screenshot 8)

```
172.18.0.1 - - [12/May/2026:21:09:41 +0000] "GET /rest/user/whoami HTTP/1.1" 200 11 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0"
172.18.0.1 - - [12/May/2026:21:09:41 +0000] "POST /rest/user/login HTTP/1.1" 500 442 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0"
172.18.0.1 - - [12/May/2026:21:09:41 +0000] "GET /rest/user/whoami HTTP/1.1" 200 11 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0"
172.18.0.1 - - [12/May/2026:21:10:02 +0000] "GET /rest/user/whoami HTTP/1.1" 304 0 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0"
172.18.0.1 - - [12/May/2026:21:10:02 +0000] "GET /rest/user/whoami HTTP/1.1" 200 11 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0"
172.18.0.1 - - [12/May/2026:21:10:02 +0000] "POST /rest/user/login HTTP/1.1" 200 799 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0"
```

(screenshot 9 demonstrates alerts of the second day of the attack as generated by Wazuh)

	timestamp	agent.name	rule.description	rule.level	rule.id
	May 12, 2026 @ 21:15:31.7...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 21:14:43.7...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 21:12:05.6...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 21:09:41.7...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 21:02:53.6...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 21:01:59.6...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122
	May 12, 2026 @ 20:49:59.5...	nginx-proxy	Web server 400 error code.	5	31101
	May 12, 2026 @ 20:49:55.5...	nginx-proxy	Web server 400 error code.	5	31101
	May 12, 2026 @ 20:48:47.5...	nginx-proxy	Web server 400 error code.	5	31101
	May 12, 2026 @ 20:29:55.4...	nginx-proxy	Web server 400 error code.	5	31101
	May 12, 2026 @ 20:28:37.4...	nginx-proxy	Web server 500 error code (Internal Error).	5	31122

On 13 May, the volume of events decreased drastically. Screenshot 10 shows a how a user logs in successfully and immediately starts a normal session flow, checking identity, basket, and product data. Right after that, the same session pulls account-related data like addresses, credit card info and wallet balance, then starts hitting admin configuration, which shows the account is operating with elevated access. The sequence is suspicious because after a normal login it rapidly accesses sensitive data and admin functions:

(Table 4 illustrates chronology of day 3 and successful retrieval of customer private and financial data)

Timestamp	Source	Event	Interpretation
13-05 17:19	nginx access.log	successful POST /rest/user/login returned HTTP 200	user successfully logged-in and established an open session
13-05 17:19	nginx access.log	multiple GET /rest/user/whoami requests returned HTTP 200	session was validated immediately after login

13-05 17:19	nginx access.log	GET /rest/basket/1 returned HTTP 200	session began interacting with account resources
13-05 17:21	nginx access.log	GET /api/Addressss returned HTTP 200	access to stored customer address information
13-05 17:21	nginx access.log	GET /api/Addressss/3 returned HTTP 200	retrieval of a specific customer address record
13-05 17:21	nginx access.log	GET /api/Deliveryys returned HTTP 200	access to customer delivery-related information
13-05 17:21	nginx access.log	GET /rest/admin/application-configuration returned HTTP 304	user interacted with administrative function of uncertain relevance
13-05 17:21	nginx access.log	GET /rest/wallet/balance returned HTTP 200	access to account financial information
13-05 17:21	nginx access.log	GET /api/Cards returned HTTP 200	access to stored payment card information associated with user accounts
13-05 17:23	Wazuh	Wazuh sends 3 HTTP 400 errors	continued unsuccessful interaction with web services, indicated further probing beyond retrieved data

(screenshot 10 shows raw logs from the chronology table)

```

172.18.0.1 - - [13/May/2026:17:19:54 +0000] "POST /rest/user/login HTTP/1.1" 200 799 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:19:54 +0000] "GET /rest/user/whoami HTTP/1.1" 200 125 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:19:54 +0000] "GET /rest/user/whoami HTTP/1.1" 200 125 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:19:54 +0000] "GET /rest/basket/1 HTTP/1.1" 200 401 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:19:54 +0000] "GET /rest/products/search?q= HTTP/1.1" 304 0 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:19:54 +0000] "GET /api/Quantitys/ HTTP/1.1" 304 0 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:31 +0000] "GET /rest/basket/1 HTTP/1.1" 304 0 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:31 +0000] "POST /api/BasketItems/ HTTP/1.1" 200 157 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:31 +0000] "GET /api/Products/427d=Wed%20May%2013%202026 HTTP/1.1" 200 591 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:31 +0000] "GET /rest/basket/1 HTTP/1.1" 200 678 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:34 +0000] "GET /rest/user/whoami HTTP/1.1" 304 0 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:34 +0000] "GET /rest/basket/1 HTTP/1.1" 304 0 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:35 +0000] "GET /assets/public/images/products/orange_juice.jpg HTTP/1.1" 200 17320 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:37 +0000] "GET /api/Addressss HTTP/1.1" 200 275 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:44 +0000] "GET /api/Addressss/3 HTTP/1.1" 200 273 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:44 +0000] "GET /api/Deliveryys HTTP/1.1" 200 266 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:48 +0000] "GET /rest/admin/application-configuration HTTP/1.1" 304 0 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:48 +0000] "GET /api/Deliveryys/1 HTTP/1.1" 200 186 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:48 +0000] "GET /rest/wallet/balance HTTP/1.1" 200 29 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"
172.18.0.1 - - [13/May/2026:17:21:48 +0000] "GET /api/Cards HTTP/1.1" 200 237 "http://localhost:3000/" "Mozilla/5.0 (X11; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0"

```

Across all three days, the pattern shows a progression from broad probing attempts (11 May), to repeated session testing and login (12 May), and finally to narrow API parameter exploration (13 May). The consistency of source IP and endpoint interaction suggests deliberate access scenario was happening over random events.

A3.3 Queries and search parameters

The search parameters were mostly used in the raw logs through terminal and Wazuh to lesser extent. The following queries/search parameters were implemented to collect and organise findings.

1. data.url: "/rest/user/login" → the purpose was to identify all login-related activity and determine whether repeated login attempts were occurring from the same IP.
2. data.url: "/rest/admin*" → the combination of the admin*- related API was used to determine the scope of abused privilege controls (the variations included admin/ administration/ administrator etc)
3. data.status: 400 OR data.status: 500 → used to identify unconventional HTTP responses that could indicate probing, badly formed requests, or backend instability caused by attack attempts.
4. data.method: POST AND data.status: 200 → used to identify successful POST requests, particularly successful login attempts in connection with modified or submitted data. Used to also look for payload.
5. data.method: DELETE → used to look for any modification attempts
6. "OR 1=1" OR "UNION SELECT" → used to look for unmistakable SQL injection indication but no result was found

A3.4 Hypothesis

Initial hypothesis considered brute force as primary attack technique due to repetitive login failures. However, in the absence of large username volume and later persistent admin access attempts, re-assessment was needed.

Therefore, based on the observation of the activity across the three-day period, the most likely scenario is that an SQL injection in the app's login page or product-related objects was implemented by IP 172.18.0.1.

The initial phase on the 11th May shows broad probing of authentication and admin endpoints, suggesting intent to search for vulnerable APIs. On the 12th May repeated interaction with `/rest/user/login` combined with inconsistent login responses indicated attempts to manipulate login behaviour which is consistent with injection testing or credential bypass.

On 13th May, the session behaviour shifts to successful account entry followed by direct access to sensitive customer data, including addresses, payment card details, and wallet balance. This is followed by requests to admin configuration functions, indicating granted elevated access.

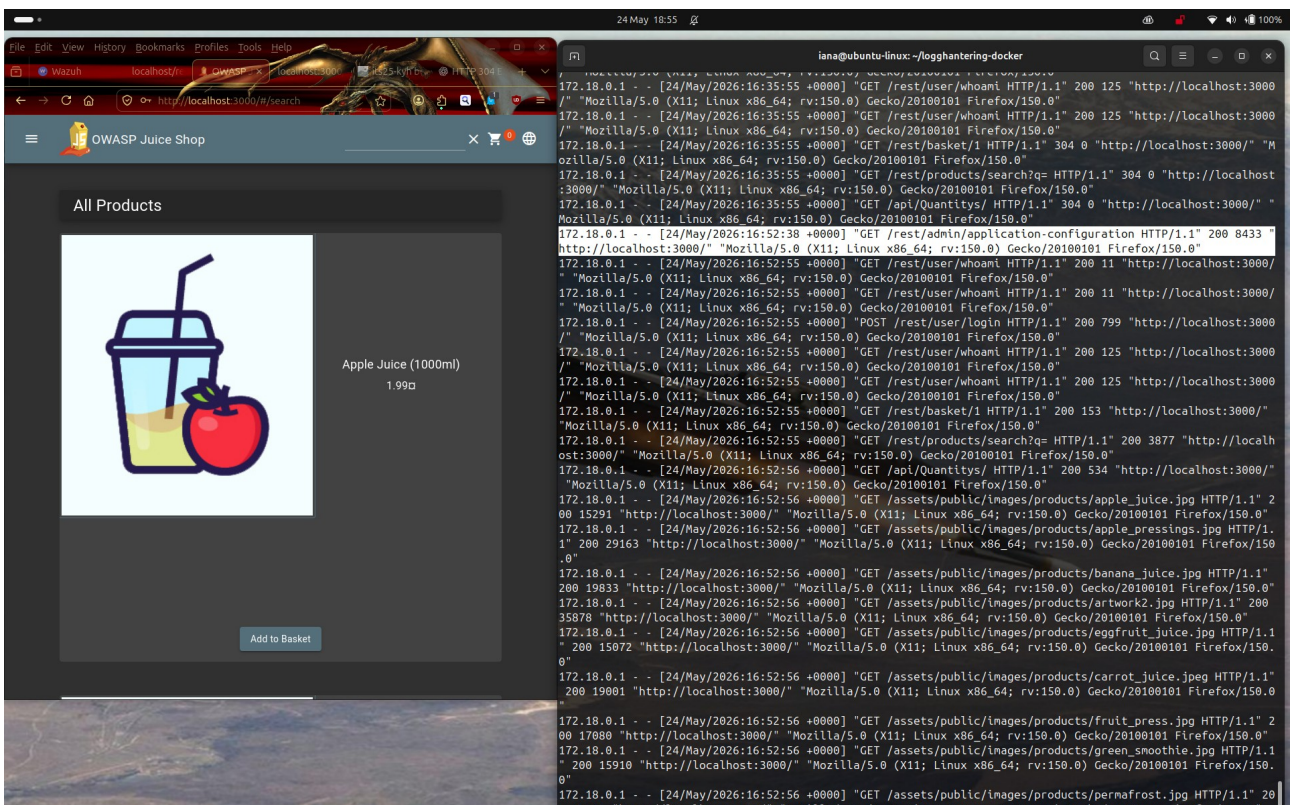
Normal user behaviour wouldn't typically involve sequential access sequentially API's such as `/rest/user/login`, `/rest/user/whoami`, `/rest/user/authentication-details`, `/rest/continue-code`, `/api/Users/*`, `/api/Users/1`, `/api/Users/2`, `/api/Users/3`, `/api/Addresss`, `/api/Addresss/3`, `/api/Cards`, `/api/Deliverys`, `/api/Deliverys/1`, `/rest/basket/1`, `/api/BasketItems`, `/api/Products/42`, `/rest/admin/application-configuration` etc. The breadth and sensitivity of the accessed resources doesn't point to standard user behaviour and is in itself enough of an evidence to start suspecting a malicious activity or potential privilege escalation going on in the background even despite lack of outright statement of attack in the log. Taken together, the pattern supports the hypothesis that SQL injection might've been used to bypass verification, resulting in access to prohibited user accounts and exposure of sensitive customer data.

The SQL injection payload may or may not have been executed- the nginx access logs didn't capture POST body contents required to directly observe the payload.

To test the behaviour of logs during a web attack, an attack was simulated against company's web service. Additionally an interactive shell has been launched to allow tracking HTTP responses upon each action in real time. Upon entering the SQL injection code (`admin' OR 1=1;--`) the page automatically transitioned to admin account and the logs returned a successful login response (**POST /rest/user/login → 200**) then started initiating multiple APIs upon successful login, including `/rest/user/whoami`, `/rest/basket/1`, and product-related endpoints.

The screenshot below shows the real-time transition from unauthenticated to authenticated (the relevant chain of events starts from the white-marked area on the console after clearing browser cache) immediately after the injection attempt. While the real-time logs don't unequivocally prove SQL injection, what they do prove beyond doubt now is it that nginx logs don't capture the actual SQL injection in any meaningful form when it does happen, and instead show what looks like a standard web traffic. In the light of the above we can only rely solely on HTTP behaviour, logic and hypothesis in reference to this particular attack.

(screenshot 11 shows the logs during the simulated SQL injection against company's web services)



A3.5 Scope assessment

The affected data scope includes users account information, individual records, customer address records, payment card details, basket contents, and account-level configuration data. Especially the evidence from 13th May shows direct access to multiple sensitive endpoints, including user identity, delivery details, wallet balance and admin-governed endpoints. This indicates that the scope of exposure extends beyond basic user data and includes privilege data accessible only after successful privilege escalation upon login.

There is no evidence in the logs provided of lateral movement beyond the affected service stack, nor signs of compromise to the company's system outside of those services already outlined. The scope is therefore limited to suspected SQL injection with potential unjustified data access.

A4. CONTAINMENT

Short-term containment

Following the awareness of suspicious access to multiple APIs, immediate containment would involve restricting further access to the affected services. This would include temporarily or permanent blocking the error-causing IP (172.18.0.1) to company's services.

All sessions associated with the suspicious activity would be closed to prevent the use of potentially corrupt tokens. Vigilance of ongoing traffic would be heightened through Wazuh and nginx logs but also other tools necessary to confirm whether the activity continues despite preventative actions.

Temporary restriction of high-risk APIs like /rest/admin/* and user personal data would also be implemented to observe the behaviour as the investigation keeps going.

Affected customer data would be treated as potentially exposed and reviewed to understand what information may have been leaked. Affected customers would be informed as soon as possible and advised on what type of data was compromised and what they should do next. Monitoring accounts or replacing payment cards may be advised as a necessary post-breach containment. The situation would also be assessed for required reporting to authorities based on the confirmed scope of the incident.

Long-term containment

Long-term containment would focus on reducing the attack surface and preventing similar exposure in the future. This includes implementing stricter access control on admin-governed endpoints through proper role-based validation on each access request.

Login verification should be reviewed so that login functionality is resistant to injection-style attempts. Sessions should be restarted/cancelled to make sure they can't be controlled, reused or turned into privileged access platform post-attack.

A4.1 Immediate response actions

Following identification of a possible attack against company's web assets, immediate containment actions were taken to prevent further spread of access, interactions with sensitive APIs. Evidence was preserved immediately.

The first response step involved continued monitoring of nginx logs through the nginx-proxy using **tail -f /var/log/nginx/access.log** in order to watch incoming requests from attacker's IP in real time and notice if the connection is still ongoing and with what result. Parallel monitoring was conducted through Wazuh where generated reports were juxtapositioned against access.log and targeted APIs.

To preserve the evidence, relevant access.log and Wazuh logs were downloaded, cataloged and hashed, then backups were made to prevent data loss. Screenshots of relevant events and entries were also taken.

As a part of investigation, all privileged APIs that were exposed through the logs were taken under particularly careful observation. This helped with noticing a pattern forming where repeated access attempts towards admin and control-related APIs were originating from the same source IP. The observed object accessed customer addresses, payment-related data, wallet balance information and admin config endpoints hence the activity was treated as potentially unwarranted access.

Company's web services and nginx container were treated as the primary affected systems for the duration of the investigation. No evidence indicates compromise outside those or lateral movement beyond the above mentioned platforms.

A4.2 Evidence preservation

For the purpose of the investigation relevant logs and telemetry were preserved immediately after initial suspicious activity was identified. Evidence collection focused primarily on nginx access.log, error.log and Wazuh alerts

The main source of evidence was `/var/log/nginx/access.log` (*docker exec -it*) inside the nginx container. Real-time monitoring was performed using `tail -f /var/log/nginx/access.log` and `/var/log/nginx/error.log` with error.log returning no results on all accounts. All historical review was conducted directly through the container shell. The logs provided timestamps, HTTP methods, targeted endpoints, response codes, IP addresses and user-agent information used throughout the investigation chronology. Additional evidence was collected from Wazuh rule definitions located under `/var/ossec/ruleset/rules`. The investigation focused on timestamps from the first to last Wazuh-generated alert between 11–13 May in order to reconstruct the progress of the activity and compare Wazuh alerts with raw logs. No forensic disk imaging was performed. Basic memory acquisition through `docker export 6726d007c82f > nginx_container.tar` was done after the final attack but yielded no evidence of SQL payload. Evidence collection was therefore confined to live container inspection and log-based analysis. For in depth analysis container filesystem access and metadata inspection were used (steps: `sudo cat /var/lib/docker/containers/6726d007c82f/6726d007c82f-json.log` and `docker inspect 6726d007c82f`) for any supporting evidence but yielded no results in terms of meaningful discovery of payloads or otherwise a successful attack. No screenshots are included at this occasion as the above procedures didn't produce any relevant artefacts beyond the logged events already documented. This however doesn't imply certainty of absence of attack.

A4.3 Evidence handling and acquisition limitations

Evidence collection was conducted using Docker's filesystem infrastructure. This included nginx logs, Wazuh-generated alerts and container files located in `/var/lib/docker/containers/` (host) as well as logs exposed within the running containers.

A container filesystem snapshot was created using `docker export 6726d007c82f > nginx_container.tar` after the final generated alert. This gave us the container's file state at that point but didn't contain any additional useful evidence related to the suspected SQL injection, payload details or any meaningful attack.

Further inspection using `docker inspect 6726d007c82f` and review of container logs provided only runtime and service information (startup events, system messages, and operational logs). No additional security-relevant artefacts were found beyond what was already visible in the nginx logs. No error logs were recorded in nginx's own error log at any point.

In practice the system only recorded web requests and responses, not what was typed into forms or the internal database queries being executed. Because of this, the analysis is based on attacker's behaviour, request patterns, status codes, and accessed endpoints primarily.

A4.5 Impact assessment

The activity observed indicates successful access to sensitive data beyond what would be expected from a standard user session. Following login, the same session was used to query user-related objects and retrieve personal information including addresses, payment card details, delivery information, and wallet balance.

In addition to user data access, the session interacted with admin-related points like `/rest/admin/application-`configuration, suggesting tinkering with functions intended for privileged roles. This combination of actions indicates a breakdown in access control allowing a single session to access multiple levels of protected data.

The impact of this behaviour is primarily data confidentiality loss. Customer personal and financial information was accessed through standard API requests without evidence of access being restricted. While no evidence of data modification was present, the scope of exposed data represents a significant security concern in the context of personal and financial information processing.

Overall, the incident shows that once initial access was obtained, the service allowed broad lateral movement across sensitive datasets without defined access borders.

A.5 ERADICATION

The root cause of the incident would be addressed by fixing insecure backend logic that could allow SQL injection or broken access control. SQL injection risks can be reduced through parametrised queries, strict server-side input validation, and removal of unsafe dynamic query building.

The code related to verification and user data access should be reviewed to ensure proper separation between normal user access and admin functions.

Any broken access control logic should be removed and replaced with proper server-side access checks.

A6 RECOVERY

After fixes are applied, the system would be brought back to normal operation with increased monitoring. Services would be restored gradually while watching logging attempts and events related to obtaining access, API requests and admin access for any suspicious behaviour.

Additional monitoring would be kept in place for a period of time to confirm the system is stable and no similar activity reappears.

A6.1 Recommendations

The investigation showed limitations in logging capability. The nginx logs captured request flow, endpoints, and responses, but didn't show POST content or backend queries. To improve future handling of such events, better logging function should be implemented, including tracking of detailed access attempts, in-depth HTTP payload assembly and login-per user monitoring. This should be done with the exception in mind so as to avoid logging personal user data.

Additional monitoring improvements should include

- alerting on repeated access failures and abnormal login patterns
- alerting on access attempts to admin functions by non-admins
- rules for detecting repeated transitions from login to sensitive data access
- clear definition of what counts as sensitive data in the organization

Although Wazuh generated alerts for most suspicious HTTP requests, proper interpretation still depended heavily on manual log analysis. Better logging depth and improved detection rules would make future investigations of similar nature faster and more reliable. Splunk would most likely provide stronger insight and flexibility into HTTP behaviour but at the expense of higher financial cost.

A7 CONCLUSION

A7.1 Technical summary

The investigation identified a multi-day pattern of suspicious interaction against QuickSilver Materials web services, originating from a single IP. Activity progressed from broad probing of access and admin objects to more targeted attainment of user-related and sensitive data.

Under the period of what can be defined as tampering, log evidence shows repeated login attempts, session validation requests, and subsequent access to endpoints containing personal and financial information. While direct evidence of the injected SQL payload isn't present due to logging limitations, the sequence of authentication behaviour and the scope of accessed resources indicate that normal access controls were likely bypassed.

Wazuh alerts and nginx logs provided sufficient insight into the chronology of the events but conclusive analysis was thwarted by the lack of deeper insight into the specific outcomes of these events. As a result, conclusions are based on the request behaviour, response codes and data access rather than explicit payload acquirement.

Overall, this scenario shows a blend of effectiveness of log-based detection in identifying parts of a suspicious activity as well as limitations of relying solely on access logs for full forensic reconstruction of events.

A7.2 Consequence to the organisation

After gaining access, the session retrieved personal details such as names, delivery addresses and account-related information. Logs show that the personal data was accessed and viewed in bulk and individually. Payment card data was exposed during attacker's interaction with the site. Administrative functions were also accessed within the same session. This creates a direct threat of data breach, unwarranted access to personal records as well as its modification. Despite the lack of verifiable evidence, it's not unreasonable to expect that the data might've been extracted and further processed beyond the scope of this investigation. The exposure of personal and financial information places the company in a clear position of potential GDPR non-compliance. Under GDPR Articles 5, 32, 33, and 34. Article 5(1)(f) requires personal data to be processed in a manner that ensures appropriate security and in case of a breach it must be reported to the relevant authorities. Customers affected must be made aware of possible breach and containment measure must follow immediately.

As a long term consequence of this incident the customers of QuickSilver Materials may be affected by data leak through wider exposure of their data to 3rd parties, fraud attempts and direct financial losses. The company may sustain financial penalty for regulatory non-compliance and potential compensation claims. In addition, loss of customer trust following the incident can lead to long-term reputational and revenue damage.

A.8 LESSONS LEARNED

What worked well

Wazuh provided effective centralised visibility across services. In this incident, it successfully aggregated nginx logs and correlated them into a coherent multi-day timeline of activity.

Strengths:

- reliable transfer of nginx access.log from the nginx-proxy container
- accurate timestamp merging across Wazuh, raw logs and the host system (the difference between host system and nginx log was 2 hours but the alerts appeared in real time)
- detection of abnormal HTTP responses (the 400/500 errors)
- ability to reconstruct attacker's behaviour over a multi-day period
- upon closer inspection Wazuh was able to detect enumeration of sensitive APIs BUT it didn't show it as an obvious manner but rather as an alert lying behind 400/500 error.

What didn't work well

The main limitation was the absence of identifiable payload and reflection of all significant events (payloads, HTTP 200) in logs.

Shortcomings:

- Wazuh rules (generic HTTP 400/500 errors) were way too vague to identify the attack technique
- no certainty that the SQL injection was the exact technique used as concrete payloads weren't logged anywhere
- heavy reliance on manual investigation in nginx raw logs to reconstruct intent

An example of that is: alerts triggered on HTTP errors but didn't distinguish between authentic login attempts vs an actual attack (whether brute force or injection) or multiple failure to login vs deliberate probing.

This dramatically increased the workload with a much smaller payoff as the final conclusion remained based on inference.

Root cause

The most likely root cause of the attack is insecure backend and insufficient access control of the APIs.

Failure chain:

- the app allowed endless attempts at /rest/user/login without effective blocking of numerous failed login attempts
- upon login the same session was able to access multiple user personal/ financial data and admin config
- logging limitations prevented confirmation of SQL injection as a main attack technique, but the access patterns follow tendencies typical for either SQL injection or broken access control

Root cause is therefore best defined as:

- weak or missing server-side verification on high urgency APIs
- insufficient preventive/locking measures and abuse detection on multiple login attempts
- lack of detection for behaviours typical for privilege escalation

Wazuh's role in the investigation

Wazuh played a central role in enabling detection, reconstruction, and scoping of the incident.

Wazuh's strengths in this case:

- initial alert generation that allowed to look into specifics
- centralised log aggregation
- reconstruction of the attack timeline
- alerts for abnormal HTTP traffic
- link to nginx logs and real-time attack vector

Wazuh's limitations in this case:

- no payload inspection
- no built-in detection for some specific attack techniques (ssh breach detected in unrelated example, SQL wans't)
- limited ability to distinguish between attack, error and normal online behaviour
- heavy dependency on outside logs to interpret meaning behind alerts

Other SIEM tools in comparison to Wazuh

SPLUNK	SENTINEL	ELASTIC SECURITY
- more powerful searches across sessions - easier detection of behavioural sequence (login → sensitive account access) - ability to pivot between log types (HTTP, auth, access, error logs)	- built-in UEBA (User and Entity Behavior Analytics) - more advanced anomaly detection on API access patterns - focused on identity-based alerting models	- endpoint and network logs are placed in a single SIEM pipeline for mapping between host and network traffic - detection rules based on ECS schema congregate web request fields such as URL, method, and status code - flexibility in HTTP activity parsing allows creation of rules that detect specific behaviours instead of only generic error-based alerts.

In comparison, Wazuh is useful for log centralisation and basic rule-based detection but less effective in detecting complex, multi-stage attacks without significant manual analysis and custom rule engineering.

Concrete developments

1. Implement API authorisation checks on every high-value endpoint
 - role-validate server-side per request
 - prevent post-login horizontal and vertical privilege escalation
2. Introduce structured verification of session logging
 - log user ID, session states and transitions especially to privileged access
 - enable forensic reconstruction of abuse patterns
3. Develop custom Wazuh rules for attack chains
 - detect and flag behaviour that moves from repeated login attempts directly to admin access
 - improve detection of multi-stage attacks beyond single-event alert
4. Enable anomaly detection for API usage
 - CRUCIAL: flag unusual access to high volatility customer data like payment, address, personal data, bank details etc.
 - define a baseline of normal user behaviour and detect deviations
5. Improve log depth and its availability (without exposing sensitive data)
 - introduce metadata for each API request (endpoint category, auth state, role context)
 - improve SIEM analytical capability and access to it without exposing 3rd persons personal data

Sources

- User Manual, Wazuh.com, <https://documentation.wazuh.com/current/user-manual/index.html>
- Docker Container exec, Docker.com, <https://docs.docker.com/reference/cli/docker/container/exec/>
- “Running Docker Without the Desktop”, Docker Community Forums, Docker.com, <https://forums.docker.com/t/running-docker-without-docker-desktop/134922>
- Docker run, exec and logs, stackoverflow.com, <https://stackoverflow.com/questions/42958854/docker-run-docker-exec-and-logs>
- plenty of minor community sites like reddit were also used for research